

Proje Ana Alanı : Yazılım

Proje Tematik Alanı : Görüntü, Ses ve Yazı Tanıma ve İşleme Teknolojileri

Proje Adı (Başlığı) : Dijital Gözüm ile gerçek zamanlı göz ve baş hareketleri kullanılarak bilgisayar kontrol yazılımı geliştirilmesi

Özet

Gelişen teknoloji, sürekli artan veri ve kaynaklar ile bilgisayar kullanımı her geçen gün daha da artmaktadır. Kullanıcılar daha etkin bilgisayar kullanımı için farklı programlar ararken bir yanda da bilgisayar ile iletişimi farkında olmadan daha etkin hale getirme çabasındadırlar. Araç kullanırken, ellerimizi kullanarak yaptığımız eylemler sırasında bilgisayar ve akıllı cihazlarımıza komut gönderememekteyiz çünkü bu cihazları kullanırken parmaklarımıza, ellerimize ve dolayısıyla kollarımıza ihtiyaç duyarız. Bireysel engellilerin teknolojiye ayak uydurmaları ve bilgisayar kullanımını kolaylaştırmak için geliştirdiğimiz bu proje aynı zamanda bilgisayar kullanımını daha etkin ve hızlı hale getirebilmek için de tasarlanmıştır. Bilgisayarların ve cihazların göz ve baş hareketleri ile kontrol edilebilmesi birçok yönden fayda sağlayacağı ve bilgisayar ile aramızdaki iletişimi sırasında komut gönderme imkânı güzel bir destekleyici araç olacaktır. Bilgisayar ile dar bir bant genişliği ile iletişim kurmaktayız. Bilgisayar ile iletişim organlarımızı kullanarak komut gönderebilmemiz ve iletişim araçlarını arttırabilmemiz bu cihazları kullanmamız esnasında performansımızı geliştireceği gibi zamandan da tasarruf sağlayacaktır. Bu çalışma ile bilgisayarlardaki webcamler kullanılarak, baş hareketleri, göz hareketleri, jest ve mimikler tespit edilerek takip edilmektedir. Göz ve baş hareketlerinin belirlenen güvenli alanlar ile kıyaslaması sağlanmakta ve görüntü işleme teknikleri kullanılarak gerçek zamanlı ölçüm yapılmaktadır. Çalışmamız, bilgisayar kullanımında baş ve göz hareketlerini kullanabilmemiz için ek giriş kaynağı olacak ve bant genişliğini arttıracaktır. Göz hareketleri bilgisayar girdilerinden çok farklı olduğundan mevcut görüntü işleme kütüphanelerine eklemeler yapılarak, iki gözün farklı kombinasyonları, baş hareketleri, jest ve mimikler dikkate alınarak çok sık kullandığımız fare ve klavye kısayollarını da içeren komutlar tanımlanmıştır. Ayrıca kullanıcıların tercih edebileceği farklı modlar bir arayüz ile kullanıcıya sunulmuştur.

Anahtar kelimeler: göz ve baş ile bilgisayar kontrolü, engelliler için bilgisayar kullanımı, görüntü işleme

Amaç

Kullanıcılar ve bilgisayarlar arasında iletişim bilgisayarı kullanma performansını doğrudan etkileyen en önemli faktördür. Kullanılan programlar seçilirken en çok dikkat edilen unsur o programın arayüzünün kullanışlı olup olmadığıdır. Bir programı aktif olarak kullanabilmek için o programın arayüzünün önemi kadar bilgisayarın hangi düzeyde pratik kullanıldığı önem arz etmektedir. Bilgisayara gönderdiğimiz komutların hızı, yapılan işin süresini de kısaltmaktadır. Arayüzü aktif kullanabilmek, girilen komutların hızıyla doğrudan bağlantılıdır. Bilgisayarı sadece klavye ve Mouse ile değil, diğer duyu organlarımızı da kullanabilmemiz bilgisayarı çok daha etkin kullanmamızı sağlayacaktır. Biz de hazırlamış olduğumuz bu proje ile göz ve baş hareketlerini, jest ve mimikleri de kullanarak bilgisayara gönderdiğimiz komutları daha etkin ve zengin olmasını sağlamaktayız. Göz ve baş hareketleri ile Bilgisayarları yönetmek, daha hızlı, daha kullanışlı ve daha doğal kullanabilmeleri için güzel bir araçtır. İcat edilen cihazların etkileşim teknikleri sınırlıdır, o cihazı kullanabilmek için ellerimizi, parmaklarımızı ve kollarımız kullanmamız gerekir. Mevcut teknolojilerde, bilgisayardan kullanıcıya yönelim, kullanıcıdan bilgisayara göre daha zordur. Biz projemiz ile tek taraflı olan bu diyalogları, çift taraflı olmasına imkân sağlıyoruz. Bilgisayarlar, kamera aracılığıyla bizden gelen hareketleri girdi medyası olarak algılayacak gelen bu verileri hızlı bir şekilde işleyerek kullanıcı ile

arasındaki bant genişliğini genişletmiş olacaktır. Göz hareketleri geleneksel bilgisayar girdilerinden çok farklı olduğundan iki gözün yapabileceği hareketler farklı kombinasyonlarla işlenerek komutlar geliştirilmiştir. İnsanoğlunun göz hareketleriyle yaptığı jest ve mimikler incelenerek küresel olarak benzer komutlar belirlenmiştir. Araştırmalarımız sonucunda bu desenler incelenerek yüksek düzeyde anlama sahip birçok göz hareketi tespit edildi. Bu duruma bir örnek verecek olursak bir duruma onay vermek için yaklaşık 1 saniye iki gözümüzü aynı anda kapatıp sonra tekrar açarız. Bilgisayarda da “Enter” tuşunun görevini görecek komut olarak bu hareket belirlenmiştir. Bu tarz kombinasyonlar göz ile bilgisayar kullanımını daha doğal ve kullanışlı hale getirilebilmesi için farklı diyaloglar oluşturulmuştur. Yapmış olduğumuz bu sistem, geleneksel tek yönlü diyalogların aksine, sürekli, paralel giriş akışları ve gerçek zamanlı yüksek etkileşimli hızlı komutlar verebilme imkânı sağlamıştır.

Projemiz sayesinde fiziksel olarak engelsiz bireyler de bilgisayarı hızlı kullanmak için tercih edebileceği gibi, engelli bireyler de tercih edecektir. Kas ve iskelet sisteminde yetersizlik, eksiklik ve fonksiyon kaybı olanlar, kol, parmak ve omurgalarında kısıklık, eksiklik, fazlalık, hareket kısıtlılığı, şekil bozukluğu, kas güçsüzlüğü, kemik hastalığı olanlar, felçliler, serebral palsi, spastikler ve spina bifida olanlar yüz kaslarını kullanabilen bireyler çağımızın en büyük teknolojik gelişmelerinden olan bilgisayarlar kullanamamaktadır. Geliştirmiş olduğumuz bu proje ile bu bireyler de rahatlıkla bilgisayara komutlar gönderebilmesi sağlanacaktır.

Giriş

İnsan duyu organları incelendiğinde, beyin ile iletişimi en hızlı olan duyu organımızın göz olduğu tespit edilmiştir. Diğer duyu organlarına oranla beyin arasındaki iletim hızı yaklaşık 10 kat daha fazladır. Eylem olarak bakıldığında diğer duyu organlarımıza oranla en hızlı sonuç üreten gözdür. Bir noktayı görebilmemiz için sadece oraya bakmamız yeterlidir. Diğer duyu organlarında bir eylemi gerçekleştirmek için süre gerekir. Bilgisayar klavyesini kullanabilmek için ellerimizi ilk olarak klavyeye yaklaştırmamız gerekir. Fareyi kontrol edebilmek için elimizle fareyi tutmamız ve istediğimiz noktaya sürüklememiz gerekir, yani eylemi yapmak için zaman gereklidir. Bunun gibi birçok sebepten dolayı gelişen teknolojilerde göz hareketlerinden yararlanarak m

kontrol son yıllarda oldukça önem kazanmıştır. Göz takibi günümüz teknolojilerinde birçok alanda kullanılmaktadır. Özellikle askeri ve tıbbi alanda kullanılmakta olan göz hareketleri ile kontrol maliyetinden dolayı çok tercih edilmemektedir. Göz hareketleri takibi, uçuş simülasyonu, yanıt ve reaksiyon analizlerinde, tıbbi araştırmalar ve rehabilitasyonda, beyin travmalarının tespitinde, bilgisayar oyunları gibi birçok alanda kullanılmaktadır.

Birçok çalışmada Göze bağlı izleme (özel kontak lens), optik izleme (göze doğrudan temas etmeyen optik) ve elektrik potansiyeli ölçümü ile izleme (gözlerin etrafına yerleştirilen elektrotlar) gibi birçok teknoloji kullanılmıştır. Bu sistemlerin bazılarında kızılötesi ışık kaynağı ve yüksek çözünürlüklü kameralar ile göz takibi yapılmaktadır. Kızılötesi aydınlatma kullanılan görüntüleme sistemi göz bebeğini tespit eder ve göz hareketlerini takip eder. 30-40 cm mesafeye konulan kızılötesi kamera RGB olarak gelen görüntüyü GRAY skalaya indirger. Bu görüntüdeki veriler hough dönüşümü, daire veya elips bulma yöntemleri kullanılarak göz bebeği ve korneası tespit edilir. Bu yöntem 11 ms sürede yaklaşık olarak 50 fps hızıyla göz hareketlerini tespit edebilmektedir. Kullanılan mevcut sistemler incelendiğinde çene dayama aparatına baş sabitlenmek suretiyle monitörde gösterilen 9 adet kalibrasyon noktasına bakılması gerekmektedir. (Durna ve Arı, 2015).

21. Yüzyılda yapay zekâ (AI) ve yapay sinir ağlarının kullanımıyla birlikte görüntü işleme tekniklerinde büyük gelişmeler yaşanmıştır. Yapay zekâ ile göz hareketlerinin tespiti ve göz ile ilgili çalışmalar büyük hız kazanmaktadır.

Görüntü işleme, bir görüntü üzerinde farklı işlemler yaparak sonuçlar çıkarma işlemine

denir. Görüntü işleme bir görüntünün özelliklerini değiştirip üzerindeki nesneleri tanımlama olarak da adlandırılmaktadır. Bir görüntü işlenmesi için öncelikle sayısallaştırılması gerekmektedir. Bir görüntünün sayısallaştırılması resim üzerinde bulunan renklerin rakamlarla adlandırılması işlemidir. Güvenlik uygulamalarında da sıkça kullanılan görüntü işleme, yüz, parmak izi ve göz irisi tanımlamaları ile birçok otomasyon geliştirilmektedir. Görüntü işleme adımları ile yapılabilen bu işlemler, birçok hazır kütüphane kullanılarak da yapılabilmektedir. Bu kütüphanelerden bazıları, OpenCV, EmguCV, AForget.NET dir. Bu ve bunun gibi kütüphaneler ile çok daha az satır kod kullanılarak görüntü işleme yapılabilmektedir.(Eldem vd., 2017) Projemizde de OpenCV kütüphaneleri sıkça kullanılmıştır. Hough Circle Transform, Hough Gradient gibi birçok yöntem kullanılmıştır.

Python yazılım dili 1990 yılında Guido Van Rossum tarafından geliştirilmiştir. Python geniş bir kitle tarafından kullanılmaktadır. Bunun sebeplerine bakacak olursak, basit ve anlaşılır bir yazılımı vardır. Nesne tabanlı ve dinamik bir dil yapısına sahiptir. C++, C# ve javaya oranla Python çok daha az satır kodla daha anlaşılır yazılabilmektedir. Python bu ve bunun gibi sebeplerden dolayı çok yaygın kullanım kitlesine ulaşmıştır. PyQt, WxPython ve Tkinter gibi grafik modülleri sayesinde arayüzlü bir çok masaüstü uygulama da geliştirilebilmektedir. (Ayvaz vd. 2016)

OpenCV (Open Source Computer Vision) görüntü işleme için kullanılan açık kaynak kodlu kütüphanedir. Intel firmasının geliştirmeye başladığı bu kütüphane Nvidia, Amd, Google gibi birçok şirket ve topluluk tarafından geliştirilmeye devam etmektedir. Açık kaynak kodlu ve BSD lisansı ile geliştirilmektedir. Bu sebeple ücretsiz olarak tüm projelerde kullanılabilir. Bağımsız bir kütüphane olması sebebiyle tüm işletim sistemlerinde çalışmaktadır. Python, C++, C, Java, Matlab, EmguCV programlama dilleriyle OpenCV uygulamaları geliştirilebilir. 2500 den fazla algoritma bulunan bu kütüphane ile makine öğrenmesi ve görüntü işleme de dâhil birçok işlem yapılabilir. Günümüzde en çok kullanılan örnekleri, plaka tanımlama, üç boyutlu görüntüler üzerinde işlem yapma, yüz tanıma, insan hareketlerinin tespiti, optik okuma gibi uygulamalardır. Yapay zekânın gelişmesinde de çok büyük katkı sağlamıştır.

Yöntem

1- Kullanılan Yazılım ve Kütüphaneler

Görüntü işleme metotları Python programlama dili ve OpenCV Kütüphanesi kullanılarak geliştirilmiştir. OpenCV, intel firması tarafından geliştirilmiş ve BSD lisansı ile ücretsiz kullanılabilen açık kaynak kodlu bir “bilgisayarla görme” kütüphanesidir. OpenCV, resim ve videoları işleyerek anlamlı veriler çıkarmak için kullanılmaktadır. NumPy (Numerical Python) kütüphanesi hesaplamaları çok hızlı bir şekilde sonuçlandırabilen gelişmiş bir matematik kütüphanesidir. Numpy python fonksiyonlarına benzer fonksiyonlar içermektedir. Fakat python fonksiyonlarına göre çok daha hızlı ve işlevseldir.

```
# python ver 3.6.12

import sys
from PyQt5.QtGui import *      # pyqt 5 ver 5.15.2
from PyQt5.QtWidgets import *
from PyQt5.QtCore import *
import cv2 # ver 4.4.0.42
import dlib # ver 19.8.1
import numpy as np # ver 1.19.2
from math import hypot
import pyautogui # ver 0.9.50
import datetime
import time
from playsound import playsound # ver 1.2.2
```

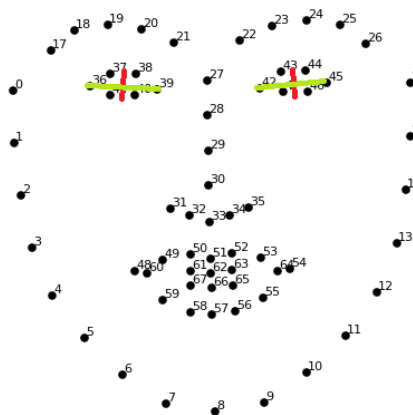
Şekil 1. Programda kullanılan kütüphaneler ve versiyonları

Görüntü işleme sırasında birden çok nesneyi ve noktayı izlemek ve işleyebilmek için dlib modülleri geliştirilmiştir. Biz de bu kütüphaneden oldukça faydalandık. Dlib kütüphanesi sayesinde yüz üzerinde 68 nokta belirlenmiştir. Bu noktalar yüz üzerinde tasarlanan sanal bir koordinat sisteminden alınan noktalardır.

PyAutoGUI kütüphanesi ile fare ve klavye kontrolü sağlanabilmektedir. Bu projede de Fare ve klavye kısayollarını kullanmak için PyAutoGUI kütüphanesinden yararlanılmıştır. Kullanıcılara işitsel dönütler verebilmek amacıyla python da en sık kullanılan kütüphanelerden biri olan playsound kütüphanesi projemize dâhil edildi. Süreli işlemler yapılması gerektiği durumlar için datetime kütüphanesi, kullanıcıların programımızı rahatlıkla kullanabilmeleri için grafiksel arayüzler tasarlanabilen PyQt5 ve yardımcı kütüphaneler kullanılmıştır. Dlib fonksiyonu python 3.6 sürümünde çalışabildiği için diğer kütüphanelerin de bu versiyonları projemize dâhil edildi. Programımızı executable (çalıştırılabilir) hale getirebilmek ve tüm bilgisayarlarda rahatlıkla kullanılabilmesi için pyinstaller kütüphanesinden yararlanılmıştır. Programın sorunsuz çalışabilmesi için ışık kaynağının yüzü karartmaya sebep olacak baş arkasında güçlü bir ışık kaynağının olmaması gerekmektedir. Aksi durumda yüz karanlıkta kalacak programımız çok istikrarlı çalışmayacaktır veya çalışmayacaktır.

2- Göz Kırpma ve Algılama

Her insanın yüzü birbirinden farklıdır. Bu sebeple dlib kütüphanesi sayesinde göz üzerinde belirlediğimiz noktalar sayesinde farklı yüzler için farklı kombinasyonlar derlenmiş hata payı ortadan kaldırılmıştır. Bu noktalar sol göz için 36,37,38,39,40,41 sağ göz için ise 42,43,44,45,46,47 olmak üzere 6 şar noktadır. Bu hatalar iki farklı fonksiyon sayesinde giderilmektedir. Bu fonksiyonlardan birincisi olan Hor ile iki gözün yatay uzunları toplamının ortalaması alınmış, (iki gözün sol ve sağ noktaları girilmektedir, 36,39,42,45) iki göz için de ayrı ayrı sol ve sağ noktalar arasındaki uzaklıklar Pisagor yöntemiyle bulunmuştur. İki göz için de farklı bir yatay uzunluk çıkması durumunda oluşabilecek hata ortadan kaldırılmıştır. İkinci fonksiyon olan Get_blink_ratio sayesinde ise bir gözün dikey uzunluğunu bulup ilk fonksiyondan çıkan yatay uzunluğa bölünerek bir oran belirlenmiş olmaktadır. Bu işlem Dlib Kütüphanesi sayesinde göz üzerinde ve ortasında bulunan bir nokta ve göz kapağında üstte ve altta ikiye nokta belirlenmektedir. Sol göz (37,38,40,41) ve sağ göz (43,44,46,47) ile alttaki ve üstteki belirlenen noktaların ortasını bulunduktan sonra bu iki nokta arasındaki uzaklık Pisagor yöntemi ile bulunup ilk fonksiyondan çıkan sonuca bölüp yuvarlama yapılır. Bu oranı kırpma algılama işlemlerinde kullanılmaktadır. İkinci fonksiyon her iki göz için de ayrı ayrı kullanılmaktadır. Çıkan sonucun ortalaması ise kırpma algılamanın ayrı ayrı yapılmasının tespitinde ve ortalama doğruluk payını arttırmak için gerekmektedir.



```
def midpoint(p1, p2):
    # girilen iki noktanın arasını bulur
    return int((p1.x + p2.x) / 2), int((p1.y + p2.y) / 2)

def hor(hor_points, facial_landmarks):
    # hor_points = [36, 39, 42, 45]
    left_left_point = (facial_landmarks.part(hor_points[0]).x, facial_landmarks.part(hor_points[0]).y)
    left_right_point = (facial_landmarks.part(hor_points[1]).x, facial_landmarks.part(hor_points[1]).y)
    right_left_point = (facial_landmarks.part(hor_points[2]).x, facial_landmarks.part(hor_points[2]).y)
    right_right_point = (facial_landmarks.part(hor_points[3]).x, facial_landmarks.part(hor_points[3]).y)
    left_hor_line_length = hypot((left_left_point[0] - left_right_point[0]),
                                (left_left_point[1] - left_right_point[1]))
    right_hor_line_length = hypot((right_left_point[0] - right_right_point[0]),
                                  (right_left_point[1] - right_right_point[1]))
    hor_line_length = (left_hor_line_length + right_hor_line_length) / 2
    return hor_line_length

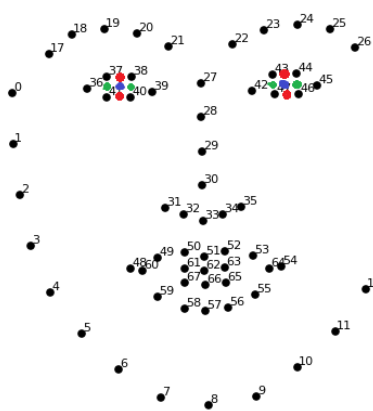
def get_blinking_ratio(eye_points, facial_landmarks):
    # sol göz = [36, 37, 38, 39, 40, 41] , sağ göz = [42, 43, 44, 45, 46, 47]
    center_top = midpoint(facial_landmarks.part(eye_points[1]), facial_landmarks.part(eye_points[2]))
    center_bottom = midpoint(facial_landmarks.part(eye_points[5]), facial_landmarks.part(eye_points[4]))
    hor_line_length = hor([36, 39, 42, 45], facial_landmarks)
    ver_line_length = hypot((center_top[0] - center_bottom[0]), (center_top[1] - center_bottom[1]))
    ratio = round(hor_line_length / ver_line_length, 1)
    return ratio
```

Şekil 2. Hor() ve get_blinking_ratio() fonksiyonlarının hesapladığı noktalar ve gösterim

Kısaca özetlemek ve şekil üzerinde belirtmek gerekirse, yatay (yeşil) çizgiler hor() fonksiyonun bulup ortalamasını aldığı çizgilerdir. Dikey (kırmızı) get_blinking_ratio() fonksiyonun bulduğu çizgi, hor() fonksiyonundan gelen yatay çizgi ortalamasını dikey çizgiye bölmek suretiyle çalışmaktadır.

3- Göz merkezinin hesaplanması

Seçenek ikide göz hareketleri ile farenin kontrolünü sağlamamız için öncelikle göz merkezinin tespit edilmesi gerekmektedir. Dlib kütüphanesinde göz merkezinde herhangi bir nokta yoktur. Fakat diğer noktalardan faydalanarak göz merkezi tespit edilebilir. Bu işlem için bir fonksiyon hazırlandı. Bir göz üzerinde altı nokta belirlenmiştir. Bu noktaların iki tanesi gözün üstüne iki tanesi de gözün altında bulunmaktadır. Bu noktalar sırasıyla sol göz için, 37,38,40,41, sağ göz için ise 43,44,46,47 dir. Üstteki noktalar olan (sağ) 37,38, (sol) 43,44, (alt) 40,41, (üst) 46,47 noktaların ortası ayrı ayrı midPoint() fonksiyonu ile tespit edilmiştir. Bu orta noktalar şekil 3 üzerinde kırmızı noktalar ile gösterilmektedir. Çıkan sonuçların sıfıncı elemanlarının ortalamaları alınmaktadır. Çıkan sonuç bize göz merkezinin x koordinatı üzerindeki konumunu vermektedir. Sol üst alt [37,41],[44,46] ve sağ üst alt [43,47],[38,40] noktalarının yine midPoint() fonksiyonu ile ortaları tespit edilmektedir. Bu orta noktalar da şekil 3 üzerinde yeşil noktalar ile gösterilmektedir. Bu aşamada da oluşan noktaların sıfıncı elemanlarının ortalamaları hesaplanmaktadır. Çıkan sonuç bize göz merkezinin y koordinatındaki konumunu vermektedir. EyeCoordinate adlı listenin sıfıncı elemanına x'i, birinci elemanına y'yi atıyoruz. Bu orta noktalar da Şekil 3 üzerinde mavi noktalar ile gösterilmektedir.



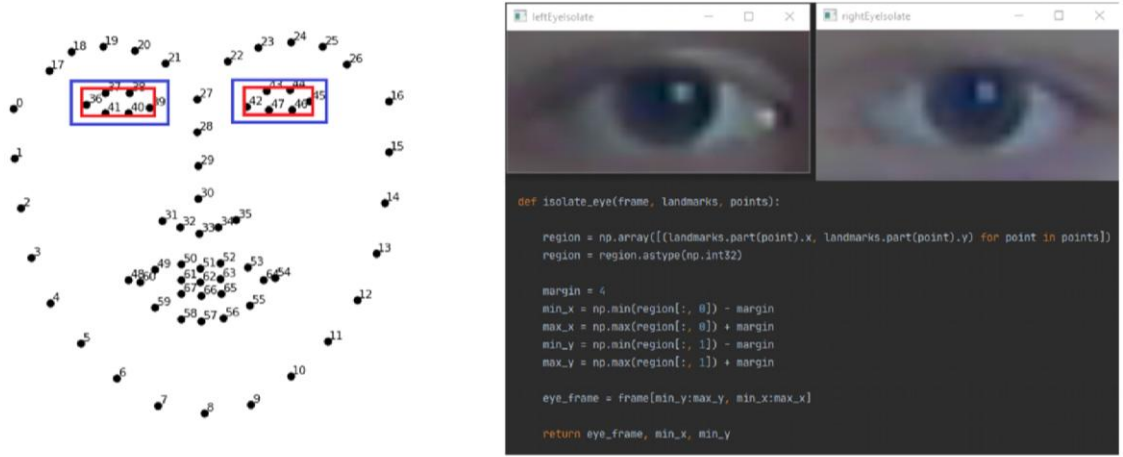
```
def eyeCenter(eye_points, facial_landmarks):  
  
    eyeXTop = ((facial_landmarks.part(eye_points[2]).x - facial_landmarks.part(  
        eye_points[1]).x) / 2 + facial_landmarks.part(eye_points[1]).x)  
    eyeXBottom = ((facial_landmarks.part(eye_points[4]).x - facial_landmarks.part(  
        eye_points[5]).x) / 2 + facial_landmarks.part(eye_points[5]).x)  
  
    eyeX = int((eyeXTop + eyeXBottom) // 2)  
  
    eyeY1 = (int((facial_landmarks.part(eye_points[5]).y - facial_landmarks.part(  
        eye_points[1]).y) // 2) + facial_landmarks.part(eye_points[1]).y)  
    eyeY2 = (int((facial_landmarks.part(eye_points[4]).y - facial_landmarks.part(  
        eye_points[2]).y) // 2) + facial_landmarks.part(eye_points[2]).y)  
  
    eyeY = int((eyeY1 + eyeY2) // 2)  
  
    eyeCoordinate = (eyeX, eyeY)  
  
    return eyeCoordinate
```

Şekil 3. Göz merkezinin hesaplanması fonksiyon ve gösterim

4- Göz bebeğinin tespit edilmesi

Seçenek ikide göz ile fare kontrolünün sağlanabilmesi, göz kırmanının algılanabilmesi ve doğruluk payını üst düzeylere çıkarılabilmesi için göz bebeğinin tespit edilmesi gerekmektedir. Dlib fonksiyonu göz bebeğinin izlemesini içermemektedir. Göz bebeğinin bulunabilmesi için göz bölgesini noktalar kullanarak izole ettikten sonra bu görüntü üzerindeki göz bebeğini bulma işlemini uygulamalıyız. Göz bebeğini diğer çevre etkilerinden izole etmek için bir fonksiyon yazıldı. Göz noktalarının bulunduğu koordinatları sol göz [36,37,38,39,40,41], sağ göz [42,43,44,45,46,47] sırası ile Numpy dizisine dönüştürdükten sonra çıkan elemanın sol sütunu noktaların x koordinatındaki yerini, sağ sütunu ise y koordinatındaki yerini vermektedir. Sol sütunun ve sağ sütunun en küçük değerleri izole görüntünün sol üst koordinatıdır. Sol ve sağ sütunların en büyük değerleri ise izole görüntünün sağ alt koordinatını vermektedir. Oluşan görüntünün hata payını önlemek için görüntü alanı büyütülmektedir. Bu yüzden en küçük

değerlerden payımızı çıkarıp en büyük değerleri ile webcam görüntüsünün belirttiğimiz aralıktaki bölümünü değişkene atıyoruz ve en küçük x ve y değerini geri döndürüyoruz. Göz merkezini gelen görüntü üzerinde bulup göz bebeğini ise izole görüntüde bulduğumuz için x ve y'ler arasında işlemler yapılamamaktadır. Bu durumda izole göz bebeğinin sol üst noktasının, görüntü üzerindeki koordinatlarını göz merkezinin koordinatlarından çıkararak işlem yapılabilir.



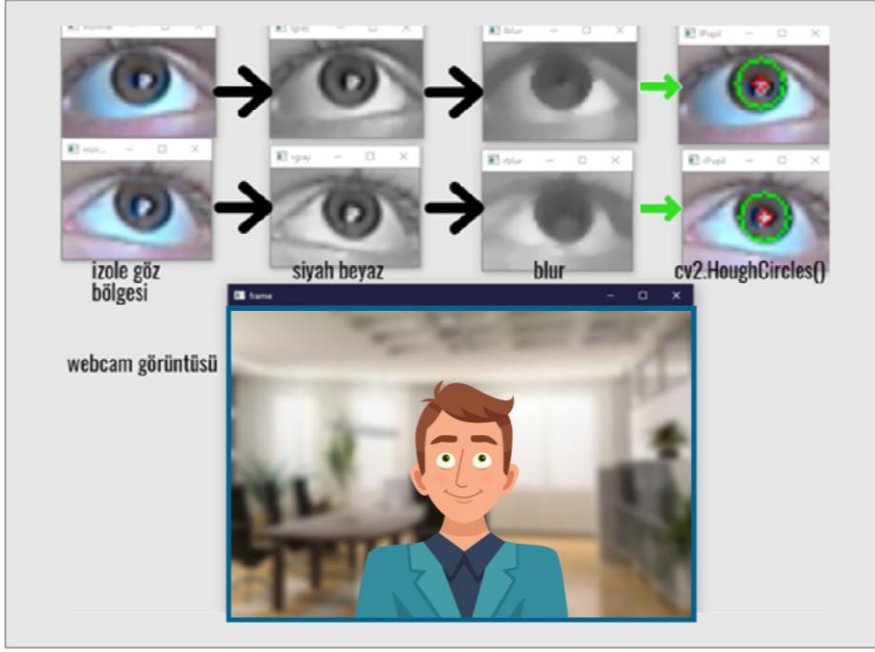
Şekil 4. İzole göz bebeğinin tespit edilmesi fonksiyon ve gösterim

Bilgisayar kullanımında göz aşağıya baktığı esnada webcamın algılayacağı açıdan uzaklaşmakta ve göz bebeğinin yönü tespit edilememektedir. Göz aşağıya bakıldığı anlarda gerçekleşmesini istediğimiz işlemler için kaşların hareketlerinden yararlanılması uygun görüldü. Ayrıca kaşların hareketleri sayesinde kısayol işlemleri tanımlandı. Oluşturduğumuz bu fonksiyon sayesinde kaşların hareketi, kalkıp olup olmadığını, kaşların orta noktasının sabit bir noktanın üzerinde olup olmadığını tespit edebiliriz. Dlib kütüphanesinde kaşlar üzerindeki noktalar bulunabilmektedir. Sol kaş için 17,18,19,20,21 ve sağ kaş için 22,23,24,25,26 olan bu noktalardan sol kaşın en sağındaki nokta 21, sağ kaşın en solundaki nokta ise 22 ile belirtilmiştir ve bu iki nokta en fazla yer değiştiren noktalardır. Bu iki nokta arasını midPoint fonksiyonu sayesinde buluyoruz. Şekil 5 üzerinde mavi renk ile gösterilen bu nokta kaşlar hareket edildiğinde en çok yer değiştiren noktadır. Sabit olarak baz aldığımız burnun en üstündeki nokta olan 27 nolu noktadır. Bu nokta kaş üzerinde belirlediğimiz orta noktanın hemen altında bulunmaktadır. Bu iki noktanın y eksenindeki koordinatları sayesinde kaşın kalkıp olup olmadığını tespit edilebilmektedir. Y ekseninden yararlanılarak hesaplanacak olan bu işlemler için uzunluktan yararlanmak gerekir. Yüz üzerindeki 0,1 ve 16,15. Noktaların y eksenindeki farklarının toplamını aldıktan sonra ortalaması bulunmalıdır. Hesaplamalar yüzün şekline ve webcame olan uzaklığına göre değişeceğinden dolayı yüz üzerindeki noktalardan faydalanılmaktadır. Hazırlamış olduğumuz fonksiyon kaş ortasındaki noktanın y'si ile sabit noktanın y'sini karşılaştırmaktadır. Kaş ortasındaki noktanın y konumu sabit noktanın y konumundan fazla ise 0, az ise 1 değerini döndürmektedir. Programımızın mod 1 seçeneğinde kısayollar için kullanılan bu fonksiyon, mod 2 de hem kısayollar için hem de fareyi aşağıya götürmek için kullanılmaktadır.



Şekil 5– Kaş hareketlerini algılamak için belirlenen noktalar ve kodlar

İzole edilen sol ve sağ göz bölgeleri içerisinde OpenCV kütüphanesinde bulunan “Hough Circles Transform” yöntemi uygulanmaktadır. Bu yöntem OpenCV tarafından tek daire algılama yöntemidir. Bu işlem için sekiz adet parametre gerekmektedir. İlk parametre giriş görüntüsüdür. İkinci parametre şu anda uygulanan tek yöntem olan HOUGH_GRADIENT’ dir. Üçüncü parametre giriş görüntüsü çözünürlüğü. Dördüncü parametrede tespit edilen dairelerin merkezleri arasındaki minimum mesafedir. Parametre çok küçük ise, gerçek bir çevreye ek olarak birden fazla komşu daire yanlışlıkla tespit edilmiş olabilir. Çok büyük bazı çevreler gözden kaçabilmektedir. Tek bir daire bulacağımız için anormal bir sayı giriyoruz. Param1 kenar algılama işleminde kullanılan Treshold değeri ve param2 HOUGH_GRADIENT için kullanılan treshold değeridir. Param1 ne kadar az olursa olsun fazla daire bulacaktır. Param2 de de aynı durum geçerlidir. Yaptığımız testler sonucunda çıkan en iyi değerleri değişkenlere atamaktayız. MaxRadius değişkenine tespit edilen en büyük çapın değerlerini, minRadius değişkenine ise en küçük çapın değerleri atanmaktadır. Bu işlemlerden gelen verileri numpy değişkenine çevirmekteyiz ve cv2.circles metodu kullanılmak suretiyle iç ve dış dairelerin görüntüleri çizdirilmektedir. Daire bulunmadığı zaman ise none çıktısı verilmektedir. Bu işlemler göz kırpmaya algılamasının da yüksek doğruluk ile çalışmasını sağlamaktadır. Daire merkezi göz bebeğinin koordinatlarını vermektedir. Bu değerler bir değişkene atanmaktadır. Mod 2 de fare kontrolünde kullanılmaktadır. Kapalı gözde ve aydınlık olmayan bir ortamda göz bebeği bulunamayacağı için hata vermektedir. Bunun önüne geçmek için ise try except içerisine alınmaktadır ve iki göz için de tüm işlemler yapılmaktadır. Bu işlemlerin çalışmasını sağlayan kodlar Ek-1 içerisinde sayfa 1’de verilmiştir. Şekil 6’da tüm aşamalar sırası ile gösterilmiştir.



Şekil 6. Göz bebeği tespit edilme adımları

5- Arayüz tasarımı ve çalışması

Arayüz tasarımında kullanılan logo ve arka planlar Corel Draw programı kullanılarak tasarlanmıştır. MainWindow oluşturulup tabwidget eklenmiştir. Tabwidget içerisinde dört tane tab eklenmiştir. Her taba Qwidget eklenmiştir. İlk tab (kamera) görseller pixmape dönüştürülerek Qlabellere eklenmiştir. Bu Qlabeller ve webcam görüntüsü dikey layout içerisine alınmaktadır. StyleSheet ile webcam görüntüsünün çerçevesi mavi renkte ayarlanmıştır. Görüntüyü her zaman kullanabilmemiz için thread kullanılmaktadır. Webcamden gelen görüntüler QImage formatına çevrildikten sonra pixmapeye çevrilerek qlabel olarak eklenmektedir. QPushButton oluşturup CancelFeed fonksiyonuna bağladık bu fonksiyon webcamden gelen görüntü güncellemesini durdurup programı kapatır kullanıcı isterse bu butonu kullanarak veya çarpıya basarak programı kapatabilir. İkinci tab (ayarlar) mod1 ve mod2 de fareyi hareket ettirirken kullanılan katsayının kullanıcının dileğince artırıp azaltması için oluşturulmuştur. Logomuz QPixmap'e çevrilip Qlabel'a eklenmiştir. Bir Qlabel oluşturup içine mod1'deki katsayı yazılmıştır. Altına yatay layout içerisinde iki tane buton konulmuştur “-“ olan connect ile push_minus1_f fonksiyonuna bağlıdır her basıldığında mod1 katsayısını en az sıfır olasıya kadar birer azaltır, azaltılmış katsayıyı setText ile Qlabel üzerine yazar. “+” connect olan push_plus1_f fonksiyonuna bağlıdır her basıldığında mod1 katsayısını maksimum yüze kadar birer artırır, artırılmış katsayıyı setText ile Qlabel üzerine yazar. Bu işlemler mod2'nin katsayısını içinde yapılır. Son olarak her şey dikey layout içerisine alınır. Üçüncü tab (kısa yollar) kullanıcı kısa yolları unutabilir burda kısa yolların ne işe yaradığı ve nasıl kullanıldığı hakkında bilgi içeren görsel ve logo QPixmap'e çevrilip Qlabellere atılır ve bunlar dikey layout içerisine alınır. Dördüncü tab (Nasıl Kullanılır) programın nasıl kullanıldığı hakkında bilgi içeren görsel ve logo QPixmap'e çevrilip Qlabellere atılır ve bunlar dikey layout içerisine alınır. Ekran arayüz tasarımının çalışmasını sağlayan kod satırları EK-1 üzerinde sayfa 5,6,7,8'den ulaşılabilir.

6- Kısayol İşlemleri

Yaptığımız testler sonucunda göz kırıldığında çıkan oran sol göz, sağ göz veya ikisinin de ortalaması alındığında bir sayıyı geçmektedir. Eğer bir göz üzerinde göz bebeği

bulunamıyor fakat diğer gözde bulunabiliyor ve diğer gözden kırma oranı daha büyük ve ortalama kırma oranını geçiyor ise o göz kapalı diğer göz açık anlamına gelmektedir. İki göz de bulunamıyor ve ortalama kırma oranını geçiyor ise iki gözün de kapalı olduğu anlaşılmaktadır. Bu durumlardan yararlanarak aşağıdaki kısayollar programlanmıştır. Bu kısayollar her iki modda da çalışmaktadır.

Programı uyku moduna alma Kısayolu

İki göz kapalı olduğu durumda bir değişken artırılır (pause_blinking) geçen saniyeyi anlamak için anlık tarih datetime ile alınıp time ile saniyeye çevirilip listeye atılır (pause_time_list) ilk eleman ilk göz kırpma zamanına denk gelirken son eleman en son göz kırpılma anına denk gelir eğer aralarındaki fark beş saniyeden küçük veya eşit ise ve göz kırpma sayısı (pause_blinking) ortalama beş saniye göz kırpma değerinden büyük veya eşit ise diğer çalışan if else bloklarını durdurmak için set dizisine (pause_set) eleman eklenir diğer if else bloklarına set uzunluğu sıfır olduğu zaman çalışır şeklinde yazacağız ki eleman eklendiği zamanki set uzunluğu bir olduğu zaman çalışmasınlar. Tekrardan kısa yolun çalışabilmesi için değişken (pause_blinking) ve liste (pause_time_list) sıfırlanır, playsound ile işitsel bildirimde “durduruldu” bulunulur. Eğer aralarındaki fark beş saniyeden büyük ise ve göz kırpma sayısı (pause_blinking) ortalama beş saniye göz kırpma değerinden küçük ise tekrardan çalışabilmesi için değişken (pause_blinking) ve liste (pause_time_list) sıfırlanır.

Programın Devam Ettirilmesi Kısayolu

İki göz kapalı olduğu durumda ve program durdurulmuşsa (pause_set'in uzunluğunun bir olma durumu) bir değişken artırılır (continue_blinking) geçen saniyeyi anlamak için anlık tarih datetime ile alınıp time ile saniyeye çevirilip listeye atılır (continue_time_list) ilk eleman ilk göz kırpma zamanına denk gelirken son eleman en son göz kırpılma anına denk gelir eğer aralarındaki fark beş saniyeden küçük veya eşit ise ve göz kırpma sayısı (continue_blinking) ortalama beş saniye göz kırpma değerinden büyük veya eşit ise durdurulmuş if else bloklarını devam ettirmek için durdurma kısayolunda uzunluğu bir yapılan pause_set dizisini sıfırlayarak diğer if else bloklarını çalışır hale getiriyoruz. Tekrardan kısa yolun çalışabilmesi için değişken (continue_blinking) ve liste (continue_time_list) sıfırlanır, playsound ile işitsel bildirimde “devam” bulunulur. Eğer aralarındaki fark beş saniyeden büyük ise ve göz kırpma sayısı (continue_blinking) ortalama beş saniye göz kırpma değerinden küçük ise tekrardan çalışabilmesi için değişken (continue_blinking) ve liste (continue_time_list) sıfırlanır.

Masaüstünü göster Kısayolu

Sol göz kapalı olduğu durumda bir değişken artırılır (desktop_blinking) geçen saniyeyi anlamak için anlık tarih datetime ile alınıp time ile saniyeye çevirilip listeye atılır (desktop_time_list) ilk eleman ilk göz kırpma zamanına denk gelirken son eleman en son göz kırpılma anına denk gelir eğer aralarındaki fark beş saniyeden küçük veya eşit ise ve göz kırpma sayısı (desktop_blinking) ortalama beş saniye göz kırpma değerinden büyük veya eşit ise Pyautogui kütüphanesi ile klavye kısayol tuşlarını ("win", "d") çalıştırıyoruz. Tekrardan kısa yolun çalışabilmesi için değişken (desktop_blinking) ve liste (desktop_time_list) sıfırlanır, playsound ile işitsel bildirimde “masaüstü” bulunulur. Eğer aralarındaki fark beş saniyeden büyük ise ve göz kırpma sayısı (desktop_blinking) ortalama beş saniye göz kırpma değerinden küçük ise tekrardan çalışabilmesi için değişken (desktop_blinking) ve liste (desktop_time_list) sıfırlanır.

Ekran Klavyesi açılması Kısayolu

Sağ göz kapalı olduğu durumda bir değişken artırılır (keyboard_blinking) geçen saniyeyi anlamak için anlık tarih datetime ile alınıp time ile saniyeye listeye atılır (keyboard_time_list) ilk eleman ilk göz kırpma zamanına denk gelirken son eleman en son göz kırpılma anına denk gelir eğer aralarındaki fark beş saniyeden küçük veya eşit ise ve göz kırpma sayısı (keyboard_blinking) ortalama beş saniye göz kırpma değerinden büyük veya eşit ise Pyautogui kütüphanesi ile klavye kısayol tuşlarını ("win", "ctrl", "o") çalıştırıyoruz. Tekrardan kısa yolun çalışabilmesi için değişken (keyboard_blinking) ve liste (keyboard_time_list) sıfırlanır, playsound ile işitsel bildirimde “klavye” bulunulur. Eğer aralarındaki fark beş saniyeden büyük ise ve göz kırpma sayısı (keyboard_blinking) ortalama beş saniye göz kırpma değerinden küçük ise tekrardan çalışabilmesi için değişken (keyboard_blinking) ve liste (keyboard_time_list) sıfırlanır.

Dosya gezgini çalıştırılması Kısayolu

İki göz kapalı, kaşlar kalkık olduğu durumda bir değişken artırılır (file_blinking) geçen saniyeyi anlamak için anlık tarih datetime ile alınıp time ile saniyeye çevrilip listeye atılır (file_time_list) ilk eleman ilk göz kırpma zamanına denk gelirken son eleman en son göz kırpılma anına denk gelir eğer aralarındaki fark beş saniyeden küçük veya eşit ise ve göz kırpma sayısı (file_blinking) ortalama beş saniye göz kırpma değerinden büyük veya eşit ise Pyautogui kütüphanesi ile klavye kısayol tuşlarını ("win", "e") çalıştırıyoruz. Tekrardan kısa yolun çalışabilmesi için değişken (file_blinking) ve liste (file_time_list) sıfırlanır, playsound ile işitsel bildirimde “dosya gezgini” bulunulur. Eğer aralarındaki fark beş saniyeden büyük ise ve göz kırpma sayısı (file_blinking) ortalama beş saniye göz kırpma değerinden küçük ise tekrardan çalışabilmesi için değişken (file_blinking) ve liste (file_time_list) sıfırlanır.

Tüm bu kısayollar iki farklı seçenek için de kullanıma sunulmaktadır. Bu kısayolların çalışmasını sağlayan kodlar EK-1 kitapçığının 2. sayfasında belirtilmiştir.

7- Program Açılışı

While döngüsü içerisinde cv2.VideoCapture() fonksiyonu ile kamera seçimini webcam olacak şekilde ayarlanması sağlandı. Read() fonksiyonu ile görüntü almaya başlanmaktadır. Read() fonksiyonu ile alınan görüntüler ters alınmaktadır. Cv2.flip() fonksiyonu ile bu görüntü çevrildikten sonra dlib.get_frontal_face_detector() fonksiyonu ile yüz bulunur. Programın başında bir bekleme süresi bulunmaktadır ve kalan süre programın arayüzünde görünmesini sağladık. Kullanıcı seçimini yaparak kimseye ihtiyaç duymadan bilgisayarını rahatlıkla kullanabilmektedir. Set dizisi oluşturularak datetime ile saniye set dizisinin içerisine tanımlandı. Belirtilen süreden bu set uzunluğu çıkarılarak kalan süre hesaplanarak programın arayüzünde yazdırılabilmektedir. Set uzunluğu bizim seçtiğimiz sayı ile eşleştiği takdirde bir değişken belirlendi. Bu bir arttırılarak süre bittikten sonra diğer aşamaya geçilmesi sağlandı. Program açılışının çalışmasını sağlayan kodlar şekil 7’de gösterilmektedir.

```

while True:
    _, frame = webcam.read()
    frame = cv2.flip(frame, 1)
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = detector(gray)

    if len(secondSet) < second:
        now = datetime.datetime.now()
        nowSecond = now.strftime("%S")
        secondSet.add(nowSecond)
        secondFrame = second - len(secondSet)
        cv2.putText(frame, f"{secondFrame}", (550, 75), font, 2, blue)

    if len(secondSet) == 11:
        secondPass += 1

    try:
        # ekrana anlık olarak bir yüz daha girmesi gibi durumlarda
        # hata verip programın çökmesini önlemek için try except içine alıyoruz
        for face in faces:

```

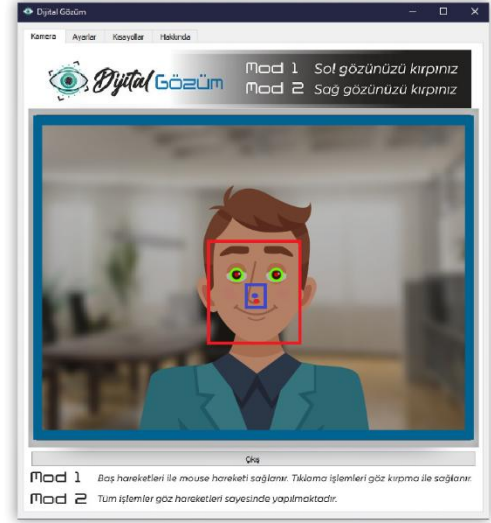
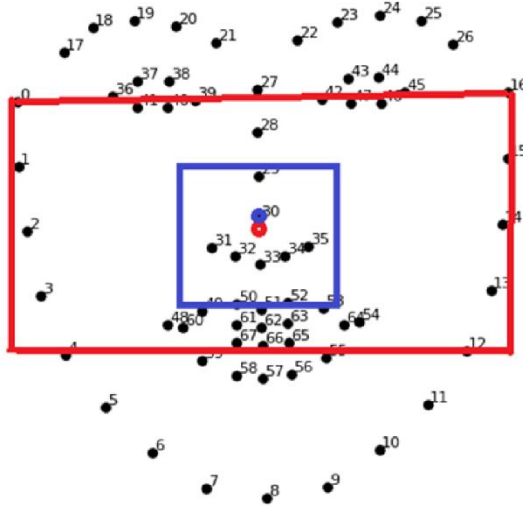
Şekil 7. Programın açılması ve seçim işleminin yapılması

8- Mod Seçme

Göz kırpma fonksiyonu sağ ve sol göz için ayrı ayrı kullanılmaktadır. Bir değişkene atılması sağlandıktan sonra ortalaması alınıp iki gözün toplam göz kırpılma oranını bulup değişkene ataması yapılmaktadır. Kaş kaldırma fonksiyonu da bir değişkene kaydedilmektedir. Başlangıç süresi bittikten sonra eğer mod seçilmediyse sol gözün ya da sağ gözün kapanması while döngüsünü devam ettirmektedir. Hangi göz kapanırsa mod seçme değişkeni artmakta ve bunun sonucunda hangi modun başlatılacağı belirlenmektedir. While döngüsünden çıkmak için mod seçme değerlerinin artıp artmadığına bakılmaktadır. Hangi değerde artış var ise o mod seçilmiş olmaktadır.

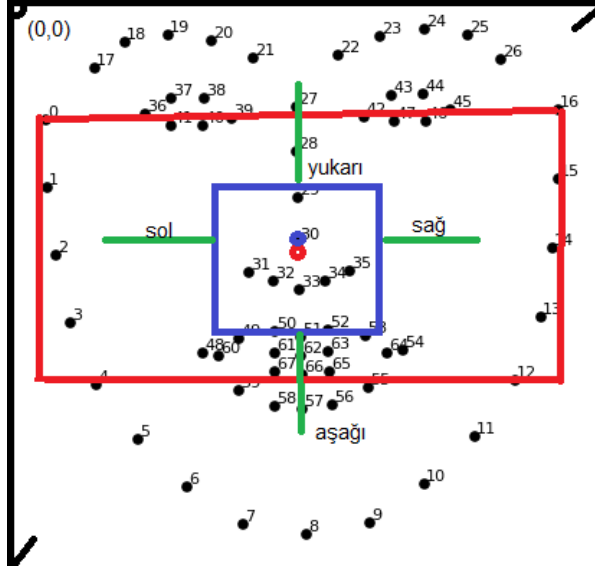
9- Mod (Seçenek) 1

Baş hareketleri ile fare kontrolü sağlamak için baş hareket ettiği anda hareket eden noktaları bulmak gerekir. Burun üzerinde belirlediğimiz en uç noktadan [30] faydalanılmıştır. X ve y koordinatlarının farkına göre işlem yapılabilmesi için yüz ortasında bir nokta belirlenmelidir. Yüzün köşeleri olarak [0,16,12] noktaları seçildi. 16. nokta ile 0. noktanın x koordinatları arasındaki fark belirlediğimiz bir sayıyla bölünüyor ve çıkan değerler 0' ın x değerinden çıkarılıp 16 nın x değerine eklenerek uzaklığı büyütülür. 12. nokta ve 0. nokta y değerleri arasında farkı belirlenen sayıya bölünmek suretiyle çıkan sonuç 0. noktanın y değerinden çıkarılıp 12. noktanın y değerine eklenmekte ve fark bulunmaktadır. Yeni noktalar kullanarak şekil 8'da gösterilen kırmızı dikdörtgen çizdirilmektedir. Bu dikdörtgen yüzümüzün tamamını alacak şekilde çizdirilmekte ve ortalanmaktadır. Noktalar arası x ve y koordinat değerlerinin farkına göre Mouse hareket ettirilebilmekte. Kullanıcının başını sabit tutmasına gerek kalmadan yanlışlıkla baş hareketleri sonucu istenmeyen fare hareketlerinin önüne geçilebilmesi için bir güvenli alan oluşturuldu. Bu güvenli alan yüzün ortasında küçük bir alandır. Burun noktası bu alandan dışarı çıktığı zaman fare hareket etmektedir. Yeni yüz noktalarından yeni 0 ve yeni 16 nın x değerleri arasındaki farkı belirlenen sayıya bölünmektedir. Çıkan sayı yüzün ortasındaki x ve y değerlerinden çıkarılmakta bu bize güvenli bölgenin sol üst köşesini vermektedir. Çıkan sayı x ve y değerine eklenmek suretiyle sağ alt köşe nokta belirlenmektedir. Oluşturulan güvenli bölge şekil 8 üzerinde mavi dikdörtgen ile gösterilmiştir.



Şekil 8. Mod 1 yüz tespiti ve güvenli alan oluşturulması

Burnun x değeri mavi dikdörtgenin sol üst noktasının x değerinden küçük ise kullanıcı kafasını sola çevirip güvenli bölgeden çıkarmış ve fareyi sola çevirmek istiyor demektir. X değerleri arasındaki fark uygun görülen bir kat sayı ile çarpılmaktadır ve farenin anlık konumunun x değerinden çıkarılmakta bunun sonucunda fare sola doğru gitmeye başlamaktadır. Eğer burnun x değeri mavi dikdörtgenin sağ alt noktasının x değerinden büyük ise kullanıcı kafasını sağa çevirip güvenli bölgeden çıkarmış olacaktır. Bunun sonucunda fare sağa hareket edecektir. X değerleri arasındaki fark bizim uygun gördüğümüz kat sayı ile çarpılmakta ve farenin anlık konumuna göre x değerine eklenir bunun sonucunda fare sağa doğru gitmektedir. Burnun y değeri mavi dikdörtgenin sol üst noktasının y değerinden küçük ise kullanıcı kafasını yukarıya çevirmek suretiyle güvenli bölgeden dışarıya çıkarmıştır. Bunun sonucunda y'ler arasındaki fark bizim belirlediğimiz bir kat sayı ile çarpılmaktadır. Farenin anlık konumundan y değeri çıkarılarak farenin yukarıya doğru gitmesi sağlanır. Eğer burnun y değeri mavi dikdörtgenin sağ alt noktasının y değerinden büyük ise kullanıcı kafasını aşağıya eğmiş ve güvenli bölgeden çıkarmıştır. Bu durumda fare aşağıya doğru hareket etmelidir. Y değerleri arasındaki fark belirlenen katsayı ile çarpılır ve farenin anlık konumunun y değerine eklenir bunun sonucunda fare yukarıya doğru hareket eder. Kullanıcı katsayıyı arayüz üzerinde "Ayarlar" bölümünden artırıp azaltarak fare hareket hızını değiştirebilir. Kullandıkça el ile kullanmadan daha hızlı hale gelebilir. Tüm kısayollar ve tıklama işlemleri kafa hareket ettirirken yanlış tıklamaları önlemek amacıyla güvenli alanda çalıştırılabilir.



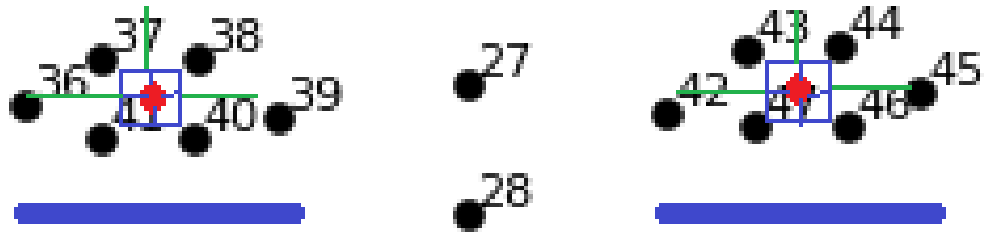
Şekil 9. Fare hareketi için gerekli koordinat eksen ve güvenli alan

Mod 1 seçeneğinin çalışmasını sağlayan programın kod satırları EK-1 üzerinde sayfa 3’de gösterilmektedir.

10- Mod (Seçenek) 2

Gözlerin göz bebeği ile çalışan bu sistemde göz bebeğinin x değerleri ile göz ortasının x değerleri işleme tabi tutularak ortalaması alınır. Bu işlemler programımızda rightlookX değişkeni ile isimlendirilmiştir. Göz bebekleri sağa baktığı zaman fareyi sağa doğru hareket ettirir. Göz bebeğinin ortadaki konumdan uzaklığı aynı zamanda fare hızını etkilen katsayıdır. Göz ortalarının x değerleri ile göz bebeklerinin x değerleri işleme tabi tutularak ortalaması alınır ve leftlookX isimli değişkene kaydedilir. Göz ortasının y değeri ile göz bebeğinin y değerleri arasındaki farklar ile de farenin yukarı aşağı hareketi belirlenmektedir. Gözler yukarıya baktığı zaman farenin hareket kat sayısı belirlenir. Gözler açık ve kaşlar kalkık ise fare aşağıya doğru inecektir. Göz bebeklerinin x ve y koordinat değerleri toplanıp ortalaması alınmak suretiyle değişkenlere (eye_pupil_x, eye_pupil_y) atanır. Göz ortalarının x ve y koordinat değerlerinin toplanıp ortalaması alınmasıyla değişkenlere (eye_center_x, eye_center_y) atanır.

Güvenli bölge oluşturulur. Güvenli bölge Şekil 10 üzerinde mavi dikdörtgen ile gösterilmektedir. Bu bölgedeki göz hareketleri fare hareketini etkilemeyecektir. Hor() fonksiyonu ile gözlerin yatay çizgisinin uzunluğu bulunur ve belirlenen bir değere bölünür. Çıkan sonuç göz ortalarının x ve y değerleri ile toplanır çıkan sonuç güvenli bölgenin sol alt noktasını belirlemektedir.



Şekil 10. Mod 2 için oluşturulan güvenli alan ve göz merkezi tespiti

Göz bebeklerinin y eksen değeri ortalaması güvenli bölgenin sol üst noktasının y değerinden küçük ise kullanıcı yukarıya bakmaktadır. Bu durumda fare yukarıya doğru hareket eder. uplookX değişkeni ile katsayıyı çarpmak suretiyle çıkan sonuç farenin y değerinden

çıkarılmaktadır. Bu durumda fare yukarıya doğru gitmektedir. Kaşlar kalkık ve gözler açık ise farenin eksen değeri azaltılır ve fare aşağıya doğru inmeye başlamaktadır. Göz bebeklerinin x değerlerinin ortalaması ile güvenli bölgenin sol üst noktasının x değerinden az ise kullanıcı sola bakıyordur ve farenin hareket yönü sol taraf olacaktır. leftlookX değeri ile katsayı çarpılarak farenin x değerinden çıkarılmakta ve fare sola doğru gitmektedir. Göz bebeklerinin x değerinin ortalaması güvenli bölgenin sağ alt noktasının x değerinden fazla ise kullanıcı sağa bakıyordur ve fare yönü sağ taraf olmaktadır. rightlookX değeri ile katsayı çarpılarak farenin x değerine eklenmekte ve fare sağa doğru hareket etmektedir. Mod 2 fare kullanımı bu şekilde sağlanmaktadır. Mod iki de tüm kısayollar ve tıklama işlemleri de kullanılabilir. Mod 2 seçeneğinin çalışmasını sağlayan programın kod satırları EK-1 üzerinde sayfa 3 ve sayfa 4’de gösterilmektedir.

Bulgular

Projemizde yüz ve göz ile bilgisayar kontrolleri sağlanabilmesi için kullanıcıya iki seçenek sunulmaktadır. Birinci seçenekte (mod1) kafa hareketleri ile fare kontrolü, göz kırpması ile fare tıklama ve kısayol işlemleri aktif edilmiş ve kullanıma sunulmuştur. Bu seçenek boyun kaslarını kullanabilen rahatlıkla başını hareket ettirebilen bireyler için tasarlanmıştır. İkinci seçenekte (mod2) ise tüm komutlar göz hareketleri ve göz kırpması ile yapılmaktadır. Bu seçenek genel olarak başını hareket ettiremeyen bireyler için düşünülmüş fakat isteğe göre tüm bireyler tarafından kullanılabilir. Bu mod kullanıldığında, fare özellikleri olan sağ ve sol tık, bilgisayar kontrolleri için kullanılan klavye kısayolları göz kırpmaları ve göz hareketleri ile sağlanmıştır. Sağ göz açık iken sol gözün kapatılması farenin sol tıklama işlemini gerçekleştirmektedir. Tam tersi olan durum yani sol göz açık iken sağ gözün kapatılması ise sağ tıklama görevini gerçekleştirmektedir. Program açıldığında 10 saniye içerisinde kullanıcı hangi modu seçeceğine karar verir (sol göz kırpması mod 1, sağ göz kırpması mod2) ve buna göre 10 saniye sonunda gözünü kırpıp programı kullanmaya başlar.



Şekil 11. Açılış ekran görüntüsü

1- Program Açılışı

Daha önce belirttiğimiz üzere program iki seçenekli bir yapıya sahiptir. Kullanıcılar bu modlardan birisini seçmek suretiyle program aktif olmaktadır. Bu program engelli bireyler için

düşünüldüğü için bunu fare ile tıklayarak belirlemek yerine, rahatlıkla göz hareketleri ile seçim yapabileceği şekilde tasarlandı. Bu nedenle hazırlamış olduğumuz program bilgisayar açıldığında otomatik olarak çalışmaktadır. Kullanıcıdan bağımsız bir şekilde webcam aktif olmakta ve görüntü işlenmeye başlamaktadır. Programın ilk açılış arayüzü Şekil 11’de görüldüğü gibidir. Programın nasıl kullanıldığına dair ayrıntılı bilgilere Şekil 13’de gösterilen menüden ulaşılabilmektedir.

2- Arayüz kullanımı

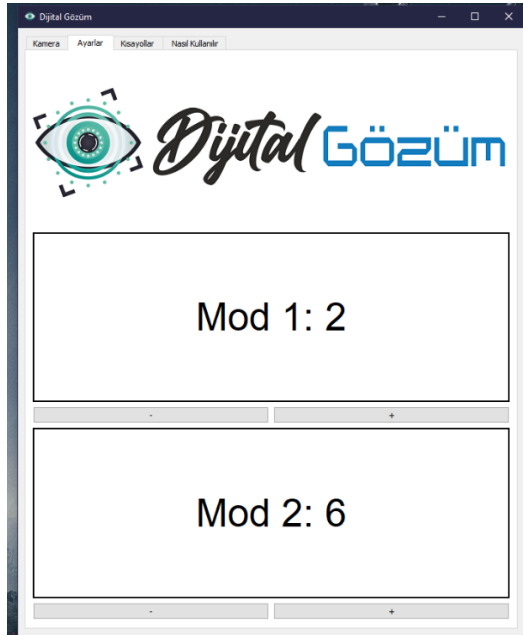
Programın arayüzünün sol üst bölgesinde dört adet tab bar bulunmaktadır (Kamera, Ayarlar, Kısa Yollar, Nasıl Kullanılır). Kamera bölümünde webcamden gelen görüntü gösterilir. Ayarlar bölümünden mod1 ve mod2 nin kat sayılarını butonlar ile artırıp azaltıp fare hareket hızını kullanıcı kendi isteğine göre değiştirebilir. Kısa yollar bölümünde kullanıcın kısa yolları unutulması durumunda tekrar hatırlaması için ne işe yarayıp nasıl çalıştıkları anlatılır. Nasıl Kullanılır bölümünde programın nasıl kullanılacağı anlatılır.

3- Mod (Seçenek) 1

Kafa hareketleri ile fare hareketi sağlanmakta göz kırpmaya ile farenin sağ, sol tık ve kısayol işlemleri yapmaktadır. Farenin hareket etmesini istediğimiz yöne doğru burnumuzun güvenli bölgeden çıkacak şekilde çeviriyoruz. Tüm kısayollar aktif.

4- Mod (Seçenek) 2

Mod 2 sadece yüz kaslarının çalışmasını gerektiren bir seçenektir. Bu modda tüm işlemler gözler ile yapılır. Mod 1 dekine benzer bir mantık kullanılmakta fakat farklı bir uygulama biçimiyle çalışmaktadır. Farenin hareket etmesini istediğimiz yöne (sol, sağ, yukarı, aşağı için gözler açıkken kaşlar kaldırılır) gözlerimiz ile bakıyoruz. Tüm kısayollar aktif.

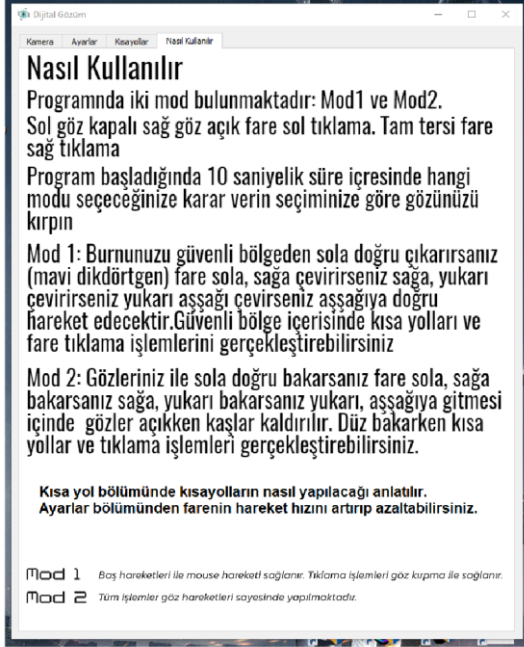


Şekil 12. Mod Ayarları Ekran Görüntüsü

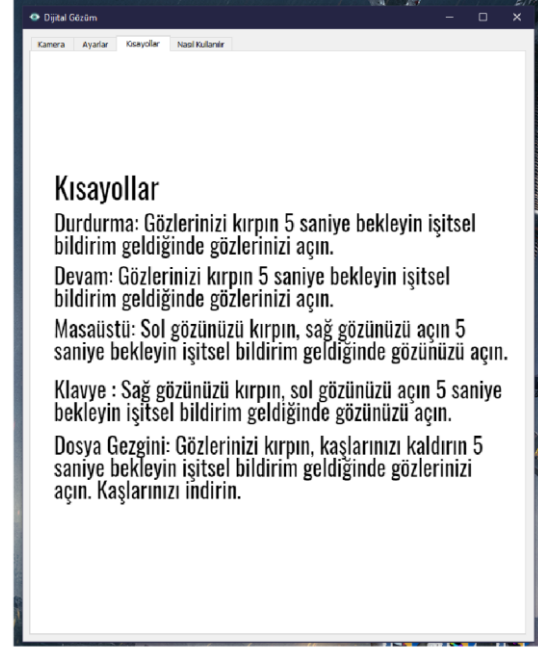
Kullanıcı katsayıyı arayüz üzerinde “Ayarlar” bölümünden artırıp azaltarak fare hareket hızını değiştirebilir. Kullanıcılar programı belirli bir süre kullandıktan sonra daha pratik kullanmaya başlayacaklardır. Mod ayarlarının değiştirilebilir olması kullanıcıların bilgisayarı daha hızlı kullanabilmesi imkânı sağlayacaktır. Şekil 12’de görüldüğü gibi kullanıcılar mod 1 ve mod 2 hız ayarları değiştirilebilmektedir.

5- Kısayol Kullanımları

Programımızın seçeneklerinden biri olan ikinci modda günlük bilgisayar kullanımında her kullanıcının ihtiyaç duyabileceği, bilgisayar ile iletişime geçmesini kolaylaştıran ve en sık kullanılan beş adet kısayol programımıza eklenmiştir. Bu kısayollar ile ilgili ayrıntı açıklamalara Şekil 14’de de görüldüğü üzere kısayollar menüsünden erişilebilmektedir.



Şekil 13. Programın nasıl kullanılır yardım ekranı



Şekil 14. Programın Kısayollar Arayüzü

Programı uyku moduna alma Kısayolu

Program denemelerinin sonucunda dizi, film, video izleme durumlarda kullanıcı uzun bir süre boyunca bilgisayar ile iletişime geçilmediği görülmüştür. Bu gibi durumlarda yanlışlıkla göz ile tıklama, kafa hareketleri ile veya göz hareketleri ile imlecin hareket ettirmesi gibi durumlar ile karşı karşıya kalınabilmektedir. Bu gibi durumların önüne geçebilmek amacıyla programı hızlı bir şekilde uyku moduna alabiliriz. Bu işlem programı kapatmaz webcam den gelen görüntüler işlemeye devam eder. Sadece fare hareketleri, tıklamalar ve devam dışındaki kısayol işlemleri durdurulur. Çalıştığı zaman kullanıcıya işitsel olarak “durduruldu” bildiriminde bulunmaktadır.

Programın Devam Ettirilmesi Kısayolu

İki göz de kapalı ve kaşlar kalkık olmadan beş saniye beklemek suretiyle devreye girmektedir. Uykuya alma işleminden sonra program tüm fonksiyonları ile çalışmaya devam edebilmesi için kullanılmaktadır. Çalıştığı zaman kullanıcıya “devam” olarak sesli bildirim vermektedir.

Masaüstünü göster Kısayolu

Bu modun çalışması için sol göz kapalı sağ göz açık kaşlar kalkık değil olacak şekilde beş saniye beklenir. Bu kısayol çalıştırıldığında çalışan tüm programlar simge durumunda küçültmek suretiyle masaüstü gösterilmektedir. Bu kısayol çalıştığı zaman işitsel olarak

“masaüstü” sesli bildirimi vermektedir.

Ekran Klavyesi açılması Kısayolu

Bu kısayolun aktif olması için sağ göz kapalı sol göz açık kaşlar kalkık değil yaklaşık beş saniye beklenmesi gerekmektedir. Kullanıcı bilgisayara kullanmak için en çok klavyeye ihtiyaç duymaktadır. Windows işletim sistemi kendi bünyesinde ekran klavyesi barındırmaktadır. Bu kısayol sayesinde ekran klavyesi kolaylıkla açılabilir. Ekran klavyesini kapatmak için de aynı komut kullanılmaktadır. Bu kısayol kullanıldığında kullanıcıya “klavye” sesli bildirimi yapmaktadır.

Dosya gezgini çalıştırılması Kısayolu

Bu kısayolun aktif olması için iki gözde kapalı kaşlar kalkık halde beş saniye beklenir. işletim sistemi üzerinde dosyalar üzerinde işlem yapmak ve dosya gezginini kullanmak gerekmektedir. Dosya gezgini eğer açık ise kapatmaktadır. Farklı klasöre gidildiyse yeni dosya gezgini açılır. Bu kısayol çalıştığında işitsel olarak “dosya gezgini” bildiriminde bulunur.

Kısayolların nasıl çalıştırılacağı ve ne işe yaradıkları arayüz üzerinde “Kısa Yollar” anlatılır kullanıcı unutursa buradan bakabilir.

Sonuç ve Tartışma

Normal hayatımızda konuşurken göz ve baş hareketlerini sıkça yaparız çünkü jest ve mimikler canlılarla iletişim kurmanın bir parçasıdır. Çoğu zaman bir iş ile ilgilenirken, bir şeyler atıştırırken, ellerimiz pis olduğunda ve bilgisayarı daha hızlı kullanmak istediğimizde yine göz ve baş hareketlerine, jest ve mimiklere ihtiyaç duyarız. Biz de bu projeye bilgisayar ile olan iletişimimizi daha etkin hale getirdik. Özellikle engelli bireylerin başkalarına ihtiyaç duymadan bilgisayarları etkin bir şekilde kullanımına olanak sağlamaktayız. Denemelerimiz ve pratik çalışmalarımız sonucunda bu programı bir oyuna alışmak gibi zamanla daha da hızlı ve aktif bir şekilde kullanabileceğimizi gözlemledik. Kısayollarda kullanılan algoritmalar zaman içinde bütünleşerek çok rahat kullanılabilir. Program orta kalitede bir webcam ile doğru ışık kullanıldığında yaklaşık %95 seviyelerinde doğruluk oranıyla çalışmaktadır. Programımız Python ile programlanarak Windows işletim sisteminde çalışabilecek şekilde tasarlandı. Çünkü Python programlama dili sürekli geliştirilen, açık kaynak kodlu, esnek ve işlevsel bir yazılımdır. Program basit işlemler sonucunda Linux, Unix, Mac işletim sistemleri için de derlenebilmektedir.

Projemizi denemelerimizde ilk kullanımlarda en çok karşılaştığımız problem kısayol kombinasyonlarını hatırlayamamamız oldu. Bu nedenle programımızın arayüzüne bir yardım düğmesi ekledik. Kullanıcılar bu yardımı kullanarak kısayolları kolaylıkla hatırlayabilmektedir. Program kullanımı sırasında bir arayüz bulunmayacağından dolayı yine yardımı kolaylıkla açabilmeleri sebebiyle basit bir kısayol ile bu yardımı açabilecekleri göz kırpmaya hareketi tanımlandı.

Projemizde geliştirdiğimiz yazılım sayesinde program otomatik açılmakta ve webcam görüntülerini işleyebilmektedir. Programımızdaki arayüz sayesinde kullanıcı programı açarken belirli yönergeleri görüp takip ederek istediği modda açabilmektedir. İster baş hareketleri ve göz hareketleri isterse sadece göz hareketleri ile bilgisayarı kullanabilmektedir. Program birçok kişi üzerinde test edildi ve sorunsuz çalışmaktadır.

Teknoloji hayatımızın her alanında etkin olarak kullanılmaktadır. Her yaşta bireyler teknolojiyi kullanmaktadır. Özellikle internet üzerindeki içeriklerin artışı, birçok şeyin dijital hale gelmesi ile bilgisayar, cep telefonu ve tablet kullanımını her geçen gün arttırmaktadır. Bu

cihazları kullanırken kollarımıza, ellerimize ve el parmaklarımıza ihtiyaç duyarız. Hayatımızın önemli bir süresini ayırdığımız bu faaliyet sırasında bu uzuvlarımızı kullanamamaktayız. Projemiz geliştirildiği ve başka platformlara da aktarıldığı düşünülürse bu uzuvlarımıza ihtiyaç duymadan çok daha kolay işlerimizi yerine getirebiliriz. Ellerimizi kullanarak bir çok psikomotor becerileri eyleme dönüştürürken aynı zamanda bilgisayar ve diğer cihazları kullanabiliriz. Bu proje hayatımızdaki verimi büyük oranda attırırken hayatımızı çok daha eğlenceli hale getirecektir.

Öneriler

Yazılımımız engelli bireyler ve bilgisayarları göz ve baş hareketleri ile daha etkin kullanmak isteyen kullanıcılar için tasarlanmıştır. Fakat bu proje çok farklı projelere ışık tutacaktır. Projemiz geliştirilerek mobil platformlara başka elektronik aletlerin çalıştırılması ve kontrolünde kullanımı sağlanabilir.

Gelişen teknolojiler ile görüntüler artık daha kolay işlenebilmektedir. En basit bilgisayar bile artık işlemleri çok hızlı bir şekilde yapabilmektedir. Fakat programımız kaliteli bir kamera ve doğru ışık ile kullanıldığında doğruluk oranı %99 seviyelerine çekilebilir.

Projemiz geliştirilerek kişiselleştirilmelidir. Kısayol işlemleri kişiselleştirilerek kendi belirleyecekleri kombinasyonlarla çalıştırılabilir. Birçok program ve oyunda tuşları, daha rahat kullanabilmemiz için değiştirir ve tekrar belirleriz. Bu yazılımda da ayarlar seçeneği eklenebilir birçok tuş kombinasyonu geliştirilip farklı görevler verilebilir. Fare hareket hızını da yine bu yöntemlerle belirlenebilir. Birçok oyun ve programda farenin hızını ayarlamamız gerekir. Fareye ve oyuna uyum sağladığımızda bu hızı arttırarak daha etkin kullanmaya çalışırız. Bu programda da fare koordinatlarının x ve y değerleri belirlenen bir kat sayı ile çarpılarak yapılmaktadır. Bu katsayıyı kullanıcıların ayarlar bölümünden artırıp azaltarak daha etkin kullanımı sağlanır.

Bu projenin başarılı olabilmesi ve geliştirilmesi bu programın kullanıcı sayısının artmasına bağlıdır. Gereken yaygınlaştırma çalışmaları yapılabilir ve engelli bireylere sunumları yapılarak daha etkin kullanımı sağlanabilir.

Kaynaklar

Arı, F. ve Durna, Y.(2015a). Labview ile Gerçek Zamanlı Göz bebeği Kornea Yansıması Takibi. 23.Sinyal İşleme ve İletişim Uygulamaları Kurultayı, 16-19 Mayıs, Malatya, 2182-2185.

Ayvaz, U., Çoban, A., Gürüler, H., Peker, M. (2016). Python Dilinin Öznelikleri, Programlama Eğitiminde ve Yazılım Dünyasındaki Yeri Bilişim Sistemleri Mühendisliği Bölümü, Muğla

Drewes, H. (2010). Eye Gaze Tracking for Human Computer Interaction. Dissertation, Munich: Ludwig-Maximilians.

Hough, P. (1962). Method and means for recognizing complex patterns. US Patent, 3069654.

Ghani Usman, M., Chaudry, S., Sohail, M., Geelani, M.N. (2013). GazePointer: A Real Time Mouse Pointer Control Implementation Based On Eye Gaze Tracking

Sreedevi, J., Reddy, M.S., Satyanarayana, B. (2020) Eyeball Movement based Cursor Control using Raspberry Pi and OpenCV

Sohail, M., Geelani, M.N., Ghani, M.U., Chaudhry, S., GazePointer: A Real-Time Mouse Pointer Control Implementation Based on Eye Gaze Tracking

Eldem, A., Eldem, H. , Palalı A. BEÜ Fen Bilimleri Dergisi 6(2), 44-48, 2017

OpenCV, Hough Circle Transform Tutorial, Erişim Tarihi : 10.07.2020,
https://docs.opencv.org/3.4/d4/d70/tutorial_hough_circle.html

OpenCV, JavaScript Hough Circle Transform Tutorial, Erişim Tarihi : 10.07.2020,
https://docs.opencv.org/master/d3/de5/tutorial_js_houghcircles.html

OpenCV, Python Hough Circle Transform Tutorial, Erişim Tarihi : 10.07.2020,
https://docs.opencv.org/master/da/d53/tutorial_py_houghcircles.html

Erişim Tarihi : 10.07.2020,
<https://opencv.org/>

Erişim Tarihi : 10.07.2020,
<http://dlib.net/>

PyAutoGUI's documentation, Erişim Tarihi : 10.07.2020,
<https://pyautogui.readthedocs.io/en/latest/>

Erişim Tarihi : 10.07.2020,
<https://numpy.org/>

PyQt5 Reference Guide, Erişim Tarihi : 01.07.2020,
<https://www.riverbankcomputing.com/static/Docs/PyQt5/>

Python datetime Library Documentation, Erişim Tarihi : 12.07.2020,
<https://docs.python.org/3/library/datetime.html>

Python time Library Documentation, Erişim Tarihi : 15.07.2020,
<https://docs.python.org/3/library/time.html>

Gaze controlled keyboard OpenCv, Erişim Tarihi : 05.08.2020,
<https://pysource.com/gaze-controlled-keyboard-opencv/>

International Journal for Research in Applied Science & Engineering Technology (IJRASET), Erişim Tarihi : 05.09.2020,
<https://www.ijraset.com/files/serve.php?FID=15922>

OpenCV, Python Image Thresholding Tutorial, Erişim Tarihi : 18.07.2020,
https://docs.opencv.org/master/d7/d4d/tutorial_py_thresholding.html

Saleem, S. M. T. , Fareed, S. ,Bibi, F. , Khan, A. ,Gohar, S. , Ashraf, H. H. , IMouse: Eyes Gesture Control System, Erişim Tarihi:21.10.2020,
https://thesai.org/Downloads/Volume9No9/Paper_68-IMouseEyes_Gesture_Control_System.pdf

Young, D. , Tunley, H. , Samuels R. , Specialised Hough Transform and Active Contour Methods for Real-Time Eye Tracking, Eriřim Tarihi:19.11.2020,
http://users.sussex.ac.uk/~davidy/eyetrack_csrp.pdf

Ekler

EK-1 Bazı kodların verildiđi bölümler